



School of Future Tech

Case Study Report

on

**Ancient Civilization Reconstruction &
Historical Intelligence Engine (ACRHIE)**

by

Pallavi Sarovar

150096725219

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion. 7. References

Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE)

1. Introduction to the Case Study

The Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE) is a Data Structures and Algorithms based system developed to assist archaeologists and historians in managing historical discoveries, excavation reports, and civilization trade networks.

Ancient civilizations are interconnected through trade, migration, cultural exchange, and artifact discoveries. Managing large amounts of archaeological information manually can be difficult and time-consuming. This case study demonstrates how Data Structures and Algorithms can be used to efficiently store, organize, search, and analyze historical data.

The system provides facilities for artifact management, excavation report processing, civilization relationship analysis, and trade route optimization. Various DSA concepts such as Hashing, Stack, Queue, Binary Search Tree (BST), Quick Sort, Binary Search, Breadth First Search (BFS), Depth First Search (DFS), and Dijkstra's Algorithm are implemented to solve real-world archaeological data management problems.

2. Problem Statement / Case Background (Abstract)

Background

Archaeologists frequently discover artifacts from different civilizations. These discoveries often reveal connections between cultures, trade routes, migration patterns, and historical events. Efficient management of such information requires structured storage and analysis techniques.

Traditional record management systems may not provide efficient mechanisms for searching artifacts, processing excavation reports, maintaining historical classifications, or analyzing civilization networks.

Abstract

This case study presents the design and implementation of the Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE). The system uses multiple Data Structures and Algorithms to organize archaeological records and analyze historical relationships.

Artifacts are stored and searched using Hash Tables and Binary Search. Historical classifications are maintained using Stack data structures, while excavation reports are processed using Queues. A Binary Search Tree is used to organize artifacts according to age. Civilization trade routes are represented using Graphs and analyzed through BFS, DFS, and Dijkstra's Algorithm.

The project demonstrates how DSA concepts can be applied to solve practical historical research and archaeological data management challenges.

3. Case Study Design

The ACRHIE system is designed as a modular architecture consisting of the following components:

1. Artifact Management

- Store archaeological artifacts.
- Search artifacts using Hashing and Binary Search.
- Sort artifacts according to significance score.
- Display artifacts according to age.

2. Classification Management

- Reclassify artifact civilizations.
- Maintain classification history.
- Support undo operations using Stack.

3. Excavation Report Processing

- Store excavation reports.
- Process reports in FIFO order using Queue.

4. Civilization Network Analysis

- Store civilization trade routes as a Graph.
- Analyze civilization connectivity using BFS.
- Analyze deep relationships using DFS.

- Find shortest trade routes using Dijkstra's Algorithm.

System Flow

Input File → Historical Intelligence Engine → Data Structures & Algorithms → Analysis & Results

4. Methods & Algorithms Technology Applied in the Case Study

1. Quick Sort

Used to rank artifacts according to their historical significance score.

Time Complexity:

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

2. Binary Search

Used to search artifacts efficiently using Artifact ID.

Time Complexity:

- $O(\log n)$

3. Hash Table

Used for instant artifact identification and retrieval.

Time Complexity:

- Average Search: $O(1)$

4. Stack

Used to maintain classification history and support undo functionality.

Time Complexity:

- Push: $O(1)$
- Pop: $O(1)$

5. Queue

Used to process excavation reports in FIFO order.

Time Complexity:

- Enqueue: $O(1)$
- Dequeue: $O(1)$

6. Binary Search Tree (BST)

Used to organize artifacts according to age.

Time Complexity:

- Average Insert/Search: $O(\log n)$

7. Breadth First Search (BFS)

Used to analyze civilization relationships level by level.

Time Complexity:

- $O(V + E)$

8. Depth First Search (DFS)

Used to explore civilization networks deeply.

Time Complexity:

- $O(V + E)$

9. Dijkstra's Algorithm

Used to determine the shortest trade route between civilizations.

Time Complexity:

- $O((V + E) \log V)$

Technology Stack Used

- Programming Language: C++
- Paradigm: Object-Oriented Programming
- Data Storage: STL Vector
- File Handling: C++ File Streams
- Development Environment: Visual Studio Code

- Version Control: Git & GitHub
-

5. Case Study Implementation Details and Snapshots

Project Structure

ACRHIE/

- data/
 - sample_input.txt
- src/
 - main.cpp
 - Engine.cpp
 - Engine.h
 - Algorithms.h
 - DataStructures.h
 - models.h

Input Data

The project reads data from sample_input.txt containing:

- Artifact Records
- Trade Routes
- Excavation Reports

Implemented Features

1. Artifact Ranking using Quick Sort.
2. Artifact Search using Hashing.
3. Artifact Search using Binary Search.
4. Artifact Classification Undo using Stack.
5. Excavation Report Processing using Queue.
6. Artifact Age Analysis using BST.
7. Civilization Connectivity Analysis using BFS.
8. Civilization Network Traversal using DFS.
9. Shortest Trade Route Calculation using Dijkstra's Algorithm.

Snapshots to Include

Insert screenshots of:

- Sorting Output
 - BST Output
 - Hash Search Output
 - Binary Search Output
 - Stack Undo Operation
 - Queue Processing
 - BFS Traversal
 - DFS Traversal
 - Dijkstra Shortest Path
-

6. Case Study Results and Conclusion

Findings

- Hashing provides efficient artifact lookup.
- Binary Search improves searching performance on sorted artifact records.
- Stack successfully maintains classification history.
- Queue ensures systematic processing of excavation reports.
- BST efficiently organizes artifacts according to age.
- BFS and DFS effectively analyze civilization relationships.
- Dijkstra's Algorithm successfully determines shortest trade routes.

Conclusion

The Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE) successfully demonstrates the application of Data Structures and Algorithms in archaeological and historical research.

The system efficiently manages artifacts, excavation reports, and civilization trade networks while showcasing practical implementations of Quick Sort, Binary Search, Hashing, Stack, Queue, BST, BFS, DFS, and Dijkstra's Algorithm.

The project fulfills all requirements of the problem statement and provides a scalable foundation for future archaeological intelligence systems.

7. References

1. C++ Documentation – <https://cplusplus.com>
2. GeeksforGeeks Data Structures and Algorithms Articles
3. Introduction to Algorithms – Cormen, Leiserson, Rivest, and Stein
4. Course Material: Data Structures and Algorithms using C++

5. Visual Studio Code Documentation
6. GitHub Documentation